



密码分析学实验报告

姓名: 邱泽辉、江李阳、王健一

学号: 202400460046、202400460042、202400460037

专业: 密码科学与技术

日期: October 3, 2026

Contents

1	实验目的	2
1.1	组员分工	2
2	实验环境	2
3	实验原理	2
3.1	差分路线	2
3.2	差分	2
3.3	CipherFour 算法	3
3.4	利用线性规划寻找差分路线	3
3.5	密钥恢复	3
3.5.1	信噪比	3
3.5.2	成功率	3
4	任务一	4
4.1	直接穷举思路	4
4.2	S 盒差分分布表	4
4.3	一轮差分传播	4
4.4	四轮动态规划	6
4.5	高概率差分的搜索	6
4.6	方法 2	7
4.7	第一轮不同	9
4.8	找到其他路线	9
5	任务二	11
6	任务三	11
6.1	实验方法	11
6.2	实验结果	12
7	任务四	13
7.1	5 轮密钥恢复攻击	13
7.1.1	代码运行结果	15
7.1.2	复杂度分析	17
8	实验总结	17
A	实验代码	18
A.1	寻找差分路线	18
B	实验报告完整地址	22

1 实验目的

1.1 组员分工

2 实验环境

- 硬件环境：13th Gen Intel(R) Core(TM) i5-13420H 处理器，12 个逻辑处理器，主频约为 2.1 GHz；内存 16 GB。
- 软件环境：Windows、MacOS, Visual Studio Code 集成开发环境。
- 编程语言：Python 3.14。

3 实验原理

3.1 差分路线

i 轮差分路线是指对输入对 (X, X') ，在 i 轮加密过程中满足对应第 j 轮的一对输出值 (Y, Y') ($1 \leq j \leq i$)，满足 $\delta X = X \oplus X' = \beta_0$ ， $\delta Y = \oplus Y' = \beta_j$ ($1 \leq j \leq i$)，则称

$$\beta_0 \rightarrow \beta_1 \rightarrow \cdots \rightarrow \beta_i$$

为一条 i 轮差分路线。假设密码算法的输入 X 和各轮密钥 K_i 的取值是相互独立且均匀分布的，对于满足输入差分为 β_0 、输出差分为 β_i (显然满足输入差分为 β_0 的明文不只一对) 的 i 轮差分路线概率为 $DP(\beta_0 \rightarrow \beta_i)$ ，其概率为各轮差分路线概率的乘积

$$DP(\beta_0 \rightarrow \beta_1 \rightarrow \cdots \rightarrow \beta_i) = \prod_{(j=1)}^i DP(\beta_{j-1} \rightarrow \beta_j)$$

注意这里差分路线是精确记录了包括起始、最终结果和中间每一种状态的一条路线。影响上述 i 轮差分路线的概率的主要因素是输入差分非零的 S 盒的个数以及输入差分非零的 S 盒对应的输出差分 (因为当前轮 S 盒的输出即为下一轮 S 盒的输入)，其中输入非零的 S 盒被称为活跃 S 盒。

3.2 差分

i 轮差分是指给定了输入差分 β_0 和 i 轮加密后的输出差分 β_i ，满足上述条件的称为 i 轮差分。与 i 轮差分路线相比，它不指定加密的中间状态。因此，对于 i 轮差分的概率计算如下：假设满足输入差分为 β_0 且 i 轮之后输出差分为 β_i 的 i 轮差分路线共 s 条，则：

$$\begin{aligned} DP(\beta_0 \xrightarrow{i \text{ 轮}} \beta_i) &= \sum_{t=1}^s DP(\beta_0 \xrightarrow{1 \text{ 轮}} \beta_1^t \rightarrow \cdots \beta_{i-1}^t \xrightarrow{1 \text{ 轮}} \beta_i) \\ &= \sum_{t=1}^s \left(\prod_{j=1}^i DP(\beta_{j-1}^t \xrightarrow{1 \text{ 轮}} \beta_j^t) \right) \end{aligned}$$

也就是说 i 轮差分的概率是由所有满足输入和输出条件的 i 轮差分路线共同组成的，它的概率是要大于等于单条差分路线的概率的，而且想要精确计算差分的概率，还是要回到计算每一条差分路线的概率上去。

3.3 CipherFour 算法

3.4 利用线性规划寻找差分路线

定义 $B_r(\Delta)$ 为经过 r 轮后, 到达差分状态 Δ 的所有路线中, 概率最大的一条的概率。对于输入差分, 把输入差分为 0 直接丢掉, 因为输入差分为 0 之后的概率都是 1, 只能找到全 0 的路线, 定义

$$B_0(\Delta) = \begin{cases} 1, & \Delta \neq 0, \\ 0, & \Delta = 0. \end{cases}$$

沿用书上的符号, 假设每一轮的差分为 β_r

假设已经知道了 $B_{r-1}(\beta_{r-1})$, 现在要计算 $B_r(\beta_r)$

从前驱 β_{r-1} 到达 β_r 的最佳完整路线概率为

$$\Pr(\beta_{r-1} \rightarrow \beta_r) = B_{r-1}(\beta_{r-1}) \cdot \Pr(\beta_{r-1}, \beta_r).$$

然后再遍历所有前驱, 就有

$$B_r(\beta_r) = \max_{\beta_{r-1}} \{B_{r-1}(\beta_{r-1}) \cdot \Pr(\beta_{r-1}, \beta_r)\}$$

这在代码上看是比较简单的, 只要遍历所有 β_{r-1} 就行, $B_{r-1}(\beta_{r-1}), \Pr(\beta_{r-1}, \beta_r)$ 知道 β_{r-1} 后可以直接查表

但是理论上的东西, 并不总是可行的, 要知道每个 β_{r-1} 可是有 2^{16} 个, 看上去也不是很多, 但是注意这里的时间复杂度是在指数上的, 也就是 $O(2^{16r})$

那么如果是 4 轮, 就会达到 2^{64} , 那这就不是一个小数目了, 所以要拆分成 4 个 S 盒

3.5 密钥恢复

3.5.1 信噪比

差分分析的密钥恢复阶段中, 正确对个数与计数器的平均计数之比称为差分分析的信噪比, 记为 S_N 。设密钥恢复中的高概率差分概率为 p , 可恢复的密钥长度为 k 比特, 采样阶段选取 m 对明文, 去噪后有 ϵm 对保存下来, 恢复密钥阶段每个方程平均求得 η 个解, 则:

$$S_N = \frac{m\dot{p}}{\frac{\epsilon m \cdot \eta}{2^k}} = \frac{p \cdot 2^k}{\epsilon \cdot \eta}$$

3.5.2 成功率

成功率即攻击恢复出来的密钥是正确密钥的概率, 记为 P_S 。设差分分析中错误密钥的计数器相互独立且均匀分布, 若差分概率为 p , 可恢复的密钥长度为 k 比特, 采样阶段选 m 对明文, 信噪比为 S_N , 计数器前 2^{k-a} 个为正确密钥的候选值, 则对足够大的 k 和 m 有:

$$P_S = \Phi\left(\frac{\sqrt{pmS_N} - \Phi^{-1}(1 - 2^{-a})}{\sqrt{S_N + 1}}\right)$$

其中 Φ 为标准正态分布函数。在以上假设下, 要保证成功率为 P_S , 需要的明文对为:

$$m = \frac{(\sqrt{S_N + 1}\Phi^{-1}(P_S) + \Phi^{-1}(1 - 2^{-a}))^2}{S_N} p^{-1}$$

4 任务一

本任务要求寻找除题目给定差分之外的高概率四轮差分。题目给出的差分为

$$(0x0, 0x0, 0x0, 0x1) \xrightarrow{4\text{轮}} (0x0, 0x0, 0x0, 0x1),$$

也可以写成

$$0x0001 \xrightarrow{4\text{轮}} 0x0001.$$

4.1 直接穷举思路

对于固定的输入差分 α ，可以遍历所有 2^{16} 个 16 比特明文 x ，构造有序明文对 $(x, x \oplus \alpha)$ ，然后分别进行四轮 CipherFour 加密。若最终密文差分为 β ，则将对对应计数加一。因此，固定输入、输出差分的概率为

$$P(\alpha \rightarrow \beta) = \frac{\#\{x : E(x) \oplus E(x \oplus \alpha) = \beta\}}{2^{16}}.$$

对于每一个非零输入差分，共有 2^{16} 个有序明文对；如果不区分明文对的顺序，则有 2^{15} 个无序明文对。若对全部 $2^{16} - 1$ 个非零输入差分都进行穷举，需要处理约 2^{32} 组明文对，计算开销较大。因此，程序使用 S 盒差分分布表（DDT）对差分进行动态规划。

4.2 S 盒差分分布表

CipherFour 使用一个 4 进 4 出的 S 盒。对于输入差分 α 和输出差分 β ，DDT 定义为

$$\text{DDT}[\alpha][\beta] = \#\{x \in \{0, 1, \dots, 15\} : S(x) \oplus S(x \oplus \alpha) = \beta\}.$$

因此，单个 S 盒的差分传播概率为

$$T[\alpha][\beta] = \frac{\text{DDT}[\alpha][\beta]}{16} = \frac{\text{DDT}[\alpha][\beta]}{2^4}.$$

代码中的 DDT 计算如下：

```
1 for diff_in in range(16):
2     for value in range(16):
3         diff_out = S[value] ^ S[value ^ diff_in]
4         diff_table[diff_in][diff_out] += 1
```

其中，`diff_table[diff_in][diff_out]` 保存输入差分为 `diff_in`、输出差分为 `diff_out` 的出现次数。

4.3 一轮差分传播

一个 16 比特差分可以拆成四个半字节：

$$\Delta = (a_0, a_1, a_2, a_3).$$

若四个 S 盒的输出差分为

$$\Gamma = (b_0, b_1, b_2, b_3),$$

则在固定当前差分的条件下，四个 S 盒同时发生该传播的概率为

$$Q(\Delta, \Gamma) = \prod_{i=0}^3 T[a_i][b_i].$$

例如，输入差分 $0x0001 = (0, 0, 0, 1)$ 时，前三个 S 盒的输入差分为 0，并且

$$T[0][0] = 1.$$

只有最后一个 S 盒产生分支。对于本实验使用的 S 盒，输入差分 1 可以传播为输出差分 1、2、5，其概率分别为 $1/2$ 、 $1/4$ 、 $1/4$ 。因此，经过 S 盒层后可能得到

$$0x0001 (1/2), \quad 0x0002 (1/4), \quad 0x0005 (1/4).$$

代码中的 `score` 保存四个半字节的联合概率分布，其形状为

$$(\text{batch}, 16, 16, 16, 16).$$

处理某一个半字节时，程序计算

$$F_{\text{new}}(\beta) = \sum_{\alpha=0}^{15} F_{\text{old}}(\alpha) T[\alpha][\beta].$$

这对应代码中的矩阵乘法：

```
1 score_matrix = moved_score.reshape(-1, 16)
2 updated_score = score_matrix @ probability_table
```

程序依次处理四个 S 盒：

```
1 for position in range(4):
2     score = update_one_sbox_distribution(
3         score, probability_table, position
4     )
```

对于一个确定的当前 16 比特差分，这一过程等价于四个 S 盒概率的乘积；当当前状态本身是多个差分状态的概率混合时，程序还会对所有前驱状态的概率进行求和，因此需要保存四个半字节的联合分布，而不能只分别保存四个边缘概率。

四个 S 盒处理完成后，将得到的 16 比特差分经过 P 置换。代码预先计算所有 16 比特状态的置换结果：

```
1 permutation = np.array(
2     [P_box(value) for value in range(STATE_SIZE)],
3     dtype=np.int32,
4 )
```

由于 P 是比特置换，满足

$$P(X) \oplus P(Y) = P(X \oplus Y),$$

因此可以直接对差分状态进行 P 置换。经过 P 置换后的差分分布作为下一轮的输入。

4.4 四轮动态规划

设初始输入差分为 α ，经过 r 轮后的差分分布为 $D_r^{(\alpha)}$ 。初始时只有输入差分本身的概率为 1：

$$D_0^{(\alpha)}(\Delta) = \begin{cases} 1, & \Delta = \alpha, \\ 0, & \Delta \neq \alpha. \end{cases}$$

代码通过以下语句完成初始化：

```
1 score[rows, nibble0, nibble1, nibble2, nibble3] = 1.0
```

记一轮四个 S 盒和 P 置换的差分转移概率为 $R(\Delta, \Delta')$ ，则每轮动态规划满足

$$D_{r+1}^{(\alpha)}(\Delta') = \sum_{\Delta} D_r^{(\alpha)}(\Delta) R(\Delta, \Delta').$$

因此，四轮后的输出差分概率为

$$D_4^{(\alpha)}(\beta) = \sum_{\Delta_1, \Delta_2, \Delta_3} R(\alpha, \Delta_1) R(\Delta_1, \Delta_2) R(\Delta_2, \Delta_3) R(\Delta_3, \beta).$$

这个求和包含了从输入差分 α 到输出差分 β 的所有差分路线。代码的外层循环对应四轮传播：

```
1 for _ in range(NUMBER_OF_ROUNDS):
2     for position in range(4):
3         score = update_one_sbox_distribution(
4             score, probability_table, position
5         )
6
7     after_sbox = score.reshape(batch_size, STATE_SIZE)
8     after_pbox[:, permutation] = after_sbox
9     score = after_pbox.reshape(
10         batch_size, 16, 16, 16, 16
11     )
```

这里每一轮的输入和输出都是 16 比特差分状态的概率分布，不是具体的明文或密文。四轮循环结束后，`score` 中保存每个输入差分经过四轮后得到所有输出差分的概率。

4.5 高概率差分的搜索

程序从 `0x0001` 到 `0xFFFF` 遍历全部 $2^{16} - 1$ 个非零输入差分，并将输入差分按 32 个一批进行处理，以降低内存开销。分批处理不会减少搜索范围。

对于每个输入差分 α ，四轮动态规划得到所有输出差分 β 的概率分布。程序使用

```
1 output_differences = np.argmax(
2     output_distribution, axis=1
3 )
```

找到该输入差分对应的最大概率输出差分。随后，再在所有输入差分对应的候选结果中选择全局最大值。

题目要求排除 $0x0001 \rightarrow 0x0001$ ，因此代码只对输入差分 `0x0001` 的输出分布执行：

```

1 candidate_distribution[
2     EXCLUDED_OUTPUT_DIFFERENCE
3 ] = 0.0

```

然后重新选择该输入差分对应的最佳输出差分。

```

PS D:\Desktop\CryptoAnalysis\experiment\experiment2\task1> uv run python .\find_best_differential.py
开始遍历全部非零输入差分，并累计各差分路线的概率...
搜索进度: 128/2048 批
搜索进度: 256/2048 批
搜索进度: 384/2048 批
搜索进度: 512/2048 批
搜索进度: 640/2048 批
搜索进度: 768/2048 批
搜索进度: 896/2048 批
搜索进度: 1024/2048 批
搜索进度: 1152/2048 批
搜索进度: 1280/2048 批
搜索进度: 1408/2048 批
搜索进度: 1536/2048 批
搜索进度: 1664/2048 批
搜索进度: 1792/2048 批
搜索进度: 1920/2048 批
搜索进度: 2048/2048 批
CipherFour算法除给定差分外概率最高的4轮差分为:
0x0001 -> 0x0010
4轮差分概率: 0.0479736328125
差分概率的2次幂表示: 2^-4.381614
PS D:\Desktop\CryptoAnalysis\experiment\experiment2\task1>

```

Figure 1: 寻找高概率差分

程序最终找到的四轮高概率差分为

$$(0x0, 0x0, 0x0, 0x1) \rightarrow (0x0, 0x0, 0x1, 0x0),$$

即

$$0x0001 \rightarrow 0x0010.$$

其基于 DDT 动态规划得到的四轮差分概率为

$$P(0x0001 \rightarrow 0x0010) = 0.0479736328125 \approx 2^{-4.381614}.$$

这里计算的是连接相同输入、输出差分的所有差分路线的概率总和，而不是某一条单独差分路线的概率。后续任务二再通过枚举具体明文对，统计该输入、输出差分在固定 CipherFour 置换网络中的实际路线和频率。

4.6 方法 2

运行给的示例程序 3.2 就找到了 0001 -> 0001 -> 0001 -> 0001 -> 0001 这条路线

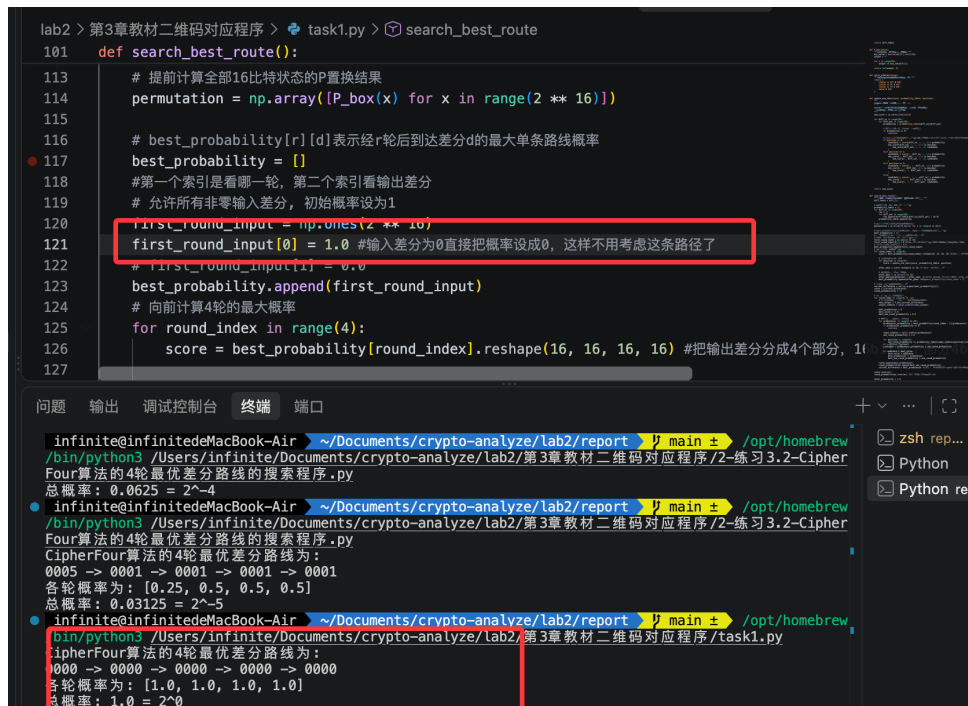
这条路线就是全局下的最优解不假，但是给的示例程序问题就是只找到了这么一条路线

不过还有可能有与他概率相等的另一条路线

一开始思路是，每一轮多保存几个差分，最后笛卡尔积，串起来，但那样好像挺麻烦，当我盯着示例程序的时候，有了一个想法

我们这里把输入差分为 0 的初始概率设成 0，就是因为它必然以概率为 1 得到输出差分为 0，所以你如果把这个也设成 1

那么最后找到的最优路线肯定是全 0



```
lab2 > 第3章教材二维码对应程序 > task1.py > search_best_route
101 def search_best_route():
113     # 提前计算全部16比特状态的P置换结果
114     permutation = np.array([P_box(x) for x in range(2 ** 16)])
115
116     # best_probability[r][d]表示经r轮后到达差分d的最大单条路线概率
117     best_probability = []
118     # 第一个索引是看哪一轮，第二个索引看输出差分
119     # 允许所有非零输入差分，初始概率设为1
120     first_round_input = np.ones(2 ** 16)
121     first_round_input[0] = 1.0 # 输入差分为0直接把概率设成0，这样不用考虑这条路径了
122     # first_round_input[1] = 0.0
123     best_probability.append(first_round_input)
124     # 向前计算4轮的最大概率
125     for round_index in range(4):
126         score = best_probability[round_index].reshape(16, 16, 16, 16) #把输出差分分成4个部分，16
127
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
1000
```

```
infinite@infiniteMacBook-Air ~/Documents/crypto-analyze/lab2/report % main % /opt/homebrew
/bin/python3 /Users/infinite/Documents/crypto-analyze/lab2/第3章教材二维码对应程序/2-练习3.2-Cipher
Four算法的4轮最优差分路线的搜索程序.py
总概率: 0.0625 = 2^-4
infinite@infiniteMacBook-Air ~/Documents/crypto-analyze/lab2/report % main % /opt/homebrew
/bin/python3 /Users/infinite/Documents/crypto-analyze/lab2/第3章教材二维码对应程序/2-练习3.2-Cipher
Four算法的4轮最优差分路线的搜索程序.py
CipherFour算法的4轮最优差分路线为:
0005 -> 0001 -> 0001 -> 0001 -> 0001
各轮概率为: [0.25, 0.5, 0.5, 0.5]
总概率: 0.03125 = 2^-5
infinite@infiniteMacBook-Air ~/Documents/crypto-analyze/lab2/report % main % /opt/homebrew
/bin/python3 /Users/infinite/Documents/crypto-analyze/lab2/第3章教材二维码对应程序/task1.py
CipherFour算法的4轮最优差分路线为:
0000 -> 0000 -> 0000 -> 0000 -> 0000
各轮概率为: [1.0, 1.0, 1.0, 1.0]
总概率: 1.0 = 2^0
```

Figure 2: 概率设成 1

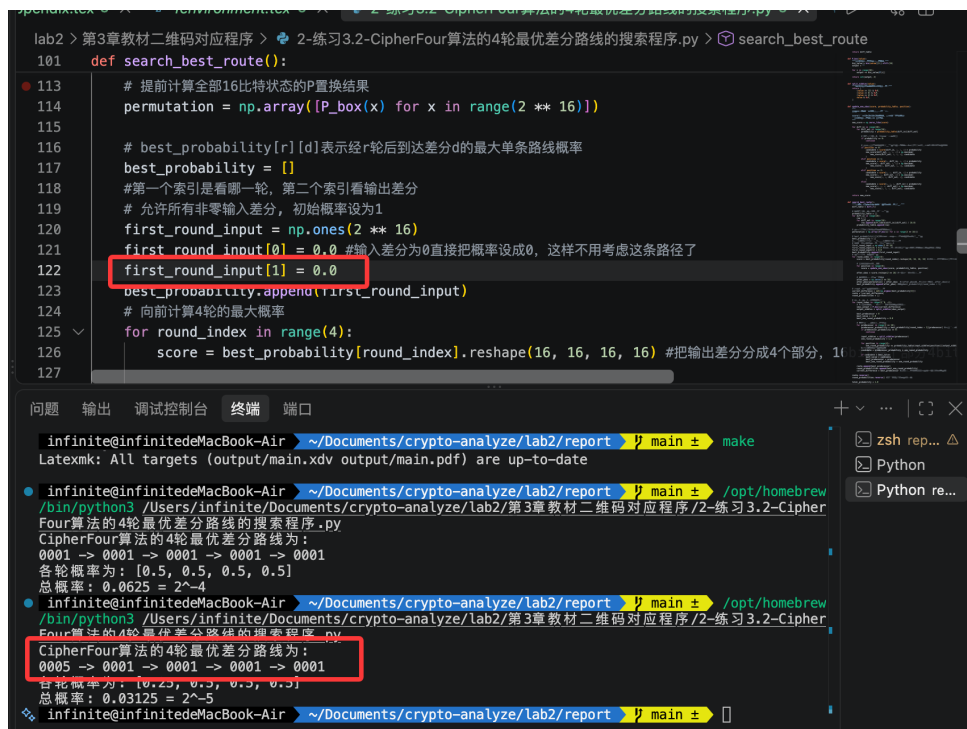
换言之，初始概率为 0 就能屏蔽这条路线，前面就是通过把输入为 0 排除掉，从而得到了 0001 这条路线

那我如果要找另一条路线，再把 0001 屏蔽掉，不就好了吗？

两个路线相同，肯定是每一步都相同，那说句废话，两个路线不同，自然就是有一步不同就行

所以我们每一轮都尝试屏蔽 0001，然后找其中概率最大的，就是题目要求的不同于 0001 这条路线的最好情况了

4.7 第一轮不同



```
lab2 > 第3章教材二维码对应程序 > 2-练习3.2-CipherFour算法的4轮最优差分路线的搜索程序.py > search_best_route
101 def search_best_route():
113     # 提前计算全部16比特状态的P置换结果
114     permutation = np.array([P_box(x) for x in range(2 ** 16)])
115
116     # best_probability[r][d]表示经r轮后到达差分d的最大单条路线概率
117     best_probability = []
118     #第一个索引是看哪一轮,第二个索引看输出差分
119     # 允许所有非零输入差分,初始概率设为1
120     first_round_input = np.ones(2 ** 16)
121     first_round_input[0] = 0.0 #输入差分为0直接把概率设成0,这样不用考虑这条路径了
122     first_round_input[1] = 0.0
123     best_probability.append(first_round_input)
124     # 向前计算4轮的最大概率
125     for round_index in range(4):
126         score = best_probability[round_index].reshape(16, 16, 16, 16) #把输出差分分成4个部分,16
127
问题 输出 调试控制台 终端 端口
infinite@infiniteMacBook-Air ~/Documents/crypto-analyze/lab2/report 主 make
Latexmk: All targets (output/main.xdv output/main.pdf) are up-to-date
infinite@infiniteMacBook-Air ~/Documents/crypto-analyze/lab2/report 主 /opt/homebrew
/bin/python3 /Users/infinite/Documents/crypto-analyze/lab2/第3章教材二维码对应程序/2-练习3.2-Cipher
Four算法的4轮最优差分路线的搜索程序.py
CipherFour算法的4轮最优差分路线为:
0001 -> 0001 -> 0001 -> 0001 -> 0001
各轮概率为: [0.5, 0.5, 0.5, 0.5]
总概率: 0.0625 = 2^-4
infinite@infiniteMacBook-Air ~/Documents/crypto-analyze/lab2/report 主 /opt/homebrew
/bin/python3 /Users/infinite/Documents/crypto-analyze/lab2/第3章教材二维码对应程序/2-练习3.2-Cipher
Four算法的4轮最优差分路线的搜索程序.py
CipherFour算法的4轮最优差分路线为:
0005 -> 0001 -> 0001 -> 0001 -> 0001
各轮概率为: [0.5, 0.5, 0.5, 0.5]
总概率: 0.03125 = 2^-5
infinite@infiniteMacBook-Air ~/Documents/crypto-analyze/lab2/report 主
```

Figure 3: 初始禁用 0001

可以看到找到了 0005 -> 0001 -> 0001 -> 0001 -> 0001 这条路线
这个不好

4.8 找到其他路线

嗯,很多时候有了开始,后面就很简单了,我们每个位置都禁用一下,给 search_best_route
加个 pos 参数,分别禁用每一次

```

lab2 > 第3章教材二维码对应程序 task1.py
101 def search_best_route(pos):
102     # 将DDT计数转换为S盒差分传播概率
103     probability_table = []
104     for diff_in in range(16):
105         row = []
106         for diff_out in range(16):
107             row.append(diff_table[diff_in][diff_out] / 16.0)
108         probability_table.append(row)
109
110     # 提前计算全部16比特状态的P置换结果
111     permutation = np.array([P_box(x) for x in range(2 ** 16)])
112
113     # best_probability[r][d]表示经r轮后到达差分d的最大单条路线概率
114     best_probability = []
115     # 第一个索引是看哪一轮, 第二个索引看输出差分
116     # 允许所有非零输入差分, 初始概率设为1
117     first_round_input = np.ones(2 ** 16)
118     first_round_input[0] = 0.0 # 输入差分为0直接把概率设成0, 这样不用考虑这条路径了
119     if pos == 0:
120         first_round_input[1] = 0.0
121     best_probability.append(first_round_input)
122
123     # 向前计算4轮的最大概率
124     for round_index in range(4):
125         score = best_probability[round_index].reshape(16, 16, 16, 16) # 把输出差分分成4个部分, 16bit每个部分4bit也就是0-15
126
127         # 依次处理4个并行S盒
128         for position in range(4):
129             score = update_one_sbox(score, probability_table, position)
130
131         after_sbox = score.reshape(2 ** 16) # 再弄回一整个输出差分
132
133         # 将S盒输出差分经过P置换
134         after_pbox = np.zeros(2 ** 16)
135         after_pbox[permutation] = after_sbox # 让after_pbox的第P[i]个位置等于after_sbox[i]
136         best_probability.append(after_pbox) # 得到best_probability[round_index + 1]
137     if round_index + 1 == pos:
138         best_probability[round_index + 1][1] = 0.0 # 禁用输入差分0001
139
140     # 找到4轮后概率最大的输出差分

```

Figure 4: 修改代码

遗憾的是, 没有和前面给的概率相等的路线, 最多只有概率为 $1/32$ 的了

```

infinite@infiniteMacBook-Air ~/Documents/crypto-analyze/lab2/report 主 main ± /opt/homebrew
/bin/python3 /Users/infinite/Documents/crypto-analyze/lab2/第3章教材二维码对应程序/task1.py
第0轮不为0001
CipherFour算法的4轮最优差分路线为:
0005 -> 0001 -> 0001 -> 0001 -> 0001
各轮概率为: [0.25, 0.5, 0.5, 0.5]
总概率: 0.03125 = 2^-5

第1轮不为0001
CipherFour算法的4轮最优差分路线为:
1000 -> 0008 -> 0001 -> 0001 -> 0001
各轮概率为: [0.5, 0.25, 0.5, 0.5]
总概率: 0.03125 = 2^-5

第2轮不为0001
CipherFour算法的4轮最优差分路线为:
0001 -> 0001 -> 0010 -> 0002 -> 1001
各轮概率为: [0.5, 0.25, 0.5, 0.375]
总概率: 0.0234375 = 2^-5

第3轮不为0001
CipherFour算法的4轮最优差分路线为:
0001 -> 0001 -> 0001 -> 0010 -> 0002
各轮概率为: [0.5, 0.5, 0.25, 0.5]
总概率: 0.03125 = 2^-5

第4轮不为0001
CipherFour算法的4轮最优差分路线为:
0001 -> 0001 -> 0001 -> 0010 -> 0002
各轮概率为: [0.5, 0.5, 0.25, 0.5]
总概率: 0.03125 = 2^-5
infinite@infiniteMacBook-Air ~/Documents/crypto-analyze/lab2/report 主 main ±

```

Figure 5: 结果

找到了五条路线, 概率都一样, 不过题目要求只看 $\Delta_{in} \rightarrow \Delta_{out}$
那么其实就是四条
0005->0001

1000->0001
0001->1001
0001->0002

5 任务二

根据任务一找到的新 4 轮高概率差分, 要寻找所有对应的差分路线, 思路为遍历所有 2^{16} 种输入差分对, 即 $xin\{0, 1, \dots, 2^{16} - 1\}$ 、 $x' = x \oplus \delta_{in}$, 记录下 4 轮加密过程中每一轮的中间异或结果, 当满足输出差分 $y \oplus y' = \Delta_{out}$ 时, 该路线即为所要求的 4 轮差分的差分路线, 将该路线记录下来, 最后输出所有差分路线并计算概率 (注意这里按照输入是有序对时共 2^{16} 种满足条件的差分对, 若按照无序对考察, 则差分路线和总的差分对数量均减半, 但最终概率不变), 计算概率为:

$$P = \frac{\#x : F^4(x) \oplus F^4(x \oplus \Delta_{in}) = \Delta_{out}}{2^{16}}$$

按照上述思路编写代码, 运行结果如下:

```

PS D:\Desktop\CryptoAnalysis\crypto-analyze\lab2\task2> ls

目录: D:\Desktop\CryptoAnalysis\crypto-analyze\lab2\task2

Mode                LastWriteTime         Length Name
----                -
-a-----          2026/9/23   11:31           3485 find_all_routes.py

PS D:\Desktop\CryptoAnalysis\crypto-analyze\lab2\task2> uv run python .\find_all_routes.py
输入差分: 0x0001
目标输出差分: 0x0010
正在计算CipherFour的一轮S盒与P置换查找表...
满足输入-输出差分的有序明文对数: 3204
不同差分路线数: 6
目标差分频率: 0.04888916015625

差分路线 | 明文对计数 | 路线频率
-----
0x0001 -> 0x0001 -> 0x0001 -> 0x0001 -> 0x0010 | 2048 | 0.0312500000
0x0001 -> 0x0001 -> 0x0001 -> 0x0002 -> 0x0010 | 562 | 0.0085750395
0x0001 -> 0x0101 -> 0x0005 -> 0x0001 -> 0x0010 | 296 | 0.0045166016
0x0001 -> 0x0001 -> 0x0101 -> 0x0005 -> 0x0010 | 238 | 0.0036315918
0x0001 -> 0x0101 -> 0x0105 -> 0x0005 -> 0x0010 | 44 | 0.0006713867
0x0001 -> 0x0101 -> 0x0050 -> 0x0002 -> 0x0010 | 16 | 0.0002441406
路线总频率为: 0.04888916015625
PS D:\Desktop\CryptoAnalysis\crypto-analyze\lab2\task2>

```

Figure 6: 所有新 4 轮差分路线

6 任务三

本任务使用五组轮密钥, 检验四轮 CipherFour 的差分概率在具体密钥下是否保持不变。输入差分和目标输出差分分别为

$$\Delta_{in} = 0x0001, \quad \Delta_{out} = 0x0010.$$

每组密钥包含五个 16 比特字: 前四个分别在四轮开始时与状态异或, 第五个在四轮结束后与状态异或。最后一个密钥字是输出白化密钥, 不是第五个 S-P 轮。

6.1 实验方法

程序先计算 16 比特状态经过一层 S 盒和 P 置换的查找表。对任一输入状态 v , 查找表保存

$$SP[v] = P(S(v)).$$

因此，对固定密钥组 $(k_0, k_1, k_2, k_3, k_4)$ ，四轮加密可写为

$$X_0 = x, \quad X_{i+1} = SP[X_i \oplus k_i] \quad (i = 0, 1, 2, 3), \quad E_k(x) = X_4 \oplus k_4.$$

代码先为全部 2^{16} 个状态构造查找表：

```
1 for value in range(STATE_SIZE):
2     SP_table[value] = P_box(S_box(value))
```

再用该表执行四轮加密：

```
1 state = plainText
2 for round_index in range(NUMBER_OF_ROUNDS):
3     state = SP_table[state ^ keys[round_index]]
4 return state ^ keys[NUMBER_OF_ROUNDS]
```

实验固定随机种子为 20260922，随机生成五组互不相同的密钥。对每组密钥，程序计算全部 2^{16} 个明文的密文，并枚举

$$(x, x') = (x, x \oplus 0x0001), \quad 0 \leq x < 2^{16}.$$

若密文差分为 $0x0010$ ，就将匹配数加一。对第 j 组密钥，记匹配数为 N_j ，则实验频率为

$$P_j = \frac{N_j}{2^{16}}.$$

循环遍历所有 x ，所以每个无序明文对按正、反两个顺序各统计一次。

6.2 实验结果

五组密钥及其统计结果如下。表中密钥字按 $(k_0, k_1, k_2, k_3, k_4)$ 排列，其中 k_4 是四轮后的输出白化密钥。

Table 1: 五组密钥下四轮差分 $0x0001 \rightarrow 0x0010$ 的统计结果

组别	密钥组 $(k_0, k_1, k_2, k_3, k_4)$	匹配对数 N_j	频率 P_j
1	[0xB8E5, 0x3463, 0x03F1, 0x26A1, 0x13E1]	3080	0.0469970703
2	[0x216F, 0xEAC8, 0xD44F, 0x1EE4, 0x6EA1]	3058	0.0466613770
3	[0x4DEC, 0x374E, 0x68DC, 0xC60D, 0x57F9]	3076	0.0469360352
4	[0x945E, 0x835B, 0x9F29, 0x33D3, 0xC82A]	3054	0.0466003418
5	[0xBC38, 0x20B4, 0x3ACF, 0x3CE6, 0xD29B]	3070	0.0468444824

五组频率的平均值为

$$\bar{P} = \frac{1}{5} \sum_{j=1}^5 P_j = 0.0468078613.$$

```

PS D:\Desktop\CryptoAnalysis\crypto-analyze\lab2\task3> ls

目录: D:\Desktop\CryptoAnalysis\crypto-analyze\lab2\task3

Mode                LastWriteTime         Length Name
----                -
-a-----         2026/9/22      19:04           3872 test_differential_frequency.py

PS D:\Desktop\CryptoAnalysis\crypto-analyze\lab2\task3> uv run python .\test_differential_frequency.py
输入差分: 0x1
目标输出差分: 0x10
密钥组数: 5
随机种子: 20260922

序号 | 轮密钥组 | 匹配对数 | 输出差分频率
-----|-----|-----|-----
1 | [0xB8E5, 0x3463, 0x03F1, 0x26A1, 0x13E1] | 3080 | 0.0469970703
2 | [0x216F, 0xEAC8, 0xD44F, 0x1EE4, 0x6EA1] | 3058 | 0.0466613770
3 | [0x4DEC, 0x374E, 0x68DC, 0xC6D0, 0x57F9] | 3076 | 0.0469360352
4 | [0x945E, 0x835B, 0x9F29, 0x33D3, 0xC82A] | 3054 | 0.0466003418
5 | [0x8C38, 0x20B4, 0x3ACF, 0x3CE6, 0xD29B] | 3070 | 0.0468444824

平均输出差分频率: 0.0468078613
PS D:\Desktop\CryptoAnalysis\crypto-analyze\lab2\task3>

```

Figure 7: 五组密钥下的差分频率统计过程与结果

五组频率落在 0.04660 到 0.04700 之间，平均约为 0.04681。任务一通过 DDT 逐轮传播差分概率，得到 0.0479736328125；本任务则对每组固定密钥穷举全部明文对，得到该具体密钥下的精确频率。两者接近但不完全相等，说明固定密钥下的逐轮差分传播并不等同于 DDT 模型给出的概率。该结果与任务一中的 DDT 差分概率传播模型、任务二中找到所有 4 轮差分路线的概率大致等，说明实际运行结果与理论分析非常相近，相互印证了前面几个实验的结果是正确的。

四轮结束后的白化密钥 k_4 对一对密文都执行相同的异或，因此在密文差分中抵消。频率随密钥组变化，来自前四轮密钥对各轮 S 盒输入状态的影响。

7 任务四

7.1 5 轮密钥恢复攻击

根据第 1) 问得到的 4 轮高概率差分路线：

$$0x0001 \longrightarrow 0x0010$$

阐述 5 轮密钥恢复攻击的基本过程：这里采用 4+1 轮模式，即在 4 轮高概率差分的末尾添加一轮。其中选择的明文对 (P, P') 满足

$$P \oplus P' = \Delta_{in} = 0x0001$$

由高概率差分可知，四轮加密后相应输出 (C_4, C'_4) 满足

$$C_4 \oplus C'_4 = \Delta_{out} = 0x0010$$

概率为 0.049，远大于随机情况的 $1/2^{16}$ 。探测其中第四轮加密结果这一中间情况的方法就是猜测第五轮的轮密钥 k_5 ：

- 若猜对 k_5 ，则由五轮加密的到的密文 (C, C') 解密回正确的 (C_4, C'_4) ，满足 $C_4 \oplus C'_4 = 0x0010$ 的概率为高概率的 0.049；
- 若猜错，则计算出来的 (C_4, C'_4) 是错误的/随机的，以随机概率满足，即 $\Pr(C_4 \oplus C'_4 = 0x0010) = \frac{1}{2^{16}}$ 。

因此基本密钥恢复方法为随机选择 m 对满足输入差分的明文对，对 k_5 的每一种可能设置一个计数器，满足：

$$S^{-1}(C \oplus k_5) \oplus S^{-1}(C' \oplus k_5) = \Delta_{out}$$

的则令计数器值 +1，处理完 m 对明文后，正确密钥的计数将明显高于错误密钥的计数，从而恢复出正确的密钥。

但是我们注意到，并非所有输入对 (P, P') 都能满足在四轮加密后满足 $C_4 \oplus C'_4 = \Delta_{out}$ ，这样不满足的对称称为错误对，恢复密钥攻击实际要使用的是正确对，错误对会对密钥恢复造成干扰，有以下事实：

- 正确对 + 正确密钥一定满足 $C_4 \oplus C'_4 = \Delta_{out}$ ；
- 正确对 + 错误密钥以随机概率满足 $C_4 \oplus C'_4 = \Delta_{out}$ ；
- 错误对 + 正确密钥一定不满足上式，即 $C_4 \oplus C'_4 \neq \Delta_{out}$ ；
- 错误对 + 错误密钥以随机概率满足 $C_4 \oplus C'_4 = \Delta_{out}$ 。

因此结合采样、去噪、恢复密钥三个步骤来筛选正确对，提高密钥恢复的成功概率。

- 采样：随机选择明文 P ，计算 $P' = P \oplus (0x0001)$ ，重复该过程 m 次，得到 m 对满足输入差分的明文对，做 5 轮 CipherFour 算法加密得到对应的密文对。我们期望得到/留下的是正确对，满足 $C_4 \oplus C'_4 = 0x0010$ ，其中发现过 S 盒子的输入 0 差分，输出还是 0 差分；且查过 S 盒的 DDT 后发现输入差分为 1 时，输出差分只可能是 1, 2, 5, 因此 5 轮加密后正确对应满足：

$$\begin{cases} \Delta C_{5,0} = \Delta C_{5,1} = \Delta C_{5,3} = 0 \\ \Delta C_{5,2} = 0x1/0x2/0x5 \end{cases}$$

- 去噪：按照上面的分析，对 m 对密文，计算器差分值 ΔC_5 ，保留满足：

$$\Delta C_5 \in \{0x0, 0x0, h, 0x0 | h \in \{0x1, 0x2, 0x5\}\}$$

注意这里保留的逻辑是，正确对一定满足上式所以被保留，特殊错误对（比如四轮加密中间值其他（第 1，2，4）字节差分为 0、第 3 半字差分并非为 0x1 的错误对也满足过 S 盒后为 h ）能满足上式而被保留，大部分错误对被排除掉，因此去噪后并不是所有留下的都是正确对，但能显著提高正确对的占比。

- 恢复密钥：设去噪后有 ϵn 对密文保留下来，记最后一轮第三个 S 盒相应的密钥为 $k_{5,2}$ ，我们的目标就是恢复这一 4bit 密钥（实际上由于其他 3 个 S 盒的输入差分都为 0 差分，因此任何密钥都能满足输入零差分到输出零差分，缺少有效信息恢复密钥其他比特），对每一对密文信息建立共 ϵn 个下述方程：

$$S^{-1}(C_{5,2}) \oplus S^{-1}(C'_{5,2} \oplus k_{5,2}) = 0x1$$

按照前面的密钥计数器方法，保留前 2^{4-a} 个作为正确密钥的候选值。对于上述方程的求解，不妨转成正向方程来看：

$$S(x \oplus k) \oplus S(x' \oplus k) = \beta$$

其中 S 为过 S 盒查表运算， (x, x', β) 已知（在这里 (x, x') 就是四轮加密中间值， β 就是五轮加密后最终密文对的差分）。一种方法是穷举 k 的所有可能，代入方程进行验证；另一种方法为借助 S 盒的差分分布表求解：记 $\oplus x' = \alpha$ ， S 盒的所有满足输入差分为 α 、输出差分为 β 的可能输入集合为 $IN_S(\alpha, \beta)$ ，得到的候选密钥集合为 $\{x \oplus X | X \in IN_S(\alpha, \beta)\}$

7.1.1 代码运行结果

首先运行明密文文件验证程序。使用实验指导书给出的六个密钥字

$(0x0123, 0x4567, 0x89AB, 0xCDEF, 0xFEDC, 0xBA98)$

这里先用上述六个轮密钥重新加密文件中的 2^{16} 个明文，计算发现密文与文件中的密文全部匹配。随后将最后一个密钥字的最后一个半字节由 $0x8$ 改为 $0x9$ ，得到 $K_6 = 0xBA99$ ，此时 2^{16} 个密文全部不匹配。这个对照确认了本地文件确实对应指导书给出的密钥组和当前采用的加密结构，为后续筛选及密钥枚举提供了匹配的实验数据。

```

PS D:\Desktop\CryptoAnalysis\crypto-analyze\lab2\task4> uv run python .\verify_reference_key_files.py
验证密钥组: ['0x0123', '0x4567', '0x89AB', '0xCDEF', '0xFEDC', '0xBA98']
明文数量: 65536
密文不匹配数: 0
验证通过: 文件中所有明密文均与该密钥组及轮结构一致。
PS D:\Desktop\CryptoAnalysis\crypto-analyze\lab2\task4> uv run python .\verify_reference_key_files.py
验证密钥组: ['0x0123', '0x4567', '0x89AB', '0xCDEF', '0xFEDC', '0xBA99']
明文数量: 65536
密文不匹配数: 65536
验证未通过: 以下为前10个不匹配项:
序号 | 明文 | 文件密文 | 按指导书密钥计算的密文
0 | 0x0000 | 0x1462 | 0x1463
1 | 0x0001 | 0x1132 | 0x1133
2 | 0x0002 | 0x1172 | 0x1173
3 | 0x0003 | 0x1477 | 0x1476
4 | 0x0004 | 0xD77C | 0xD77D
5 | 0x0005 | 0xF77E | 0xF77F
6 | 0x0006 | 0x146D | 0x146C
7 | 0x0007 | 0xF72D | 0xF72C
8 | 0x0008 | 0xD66B | 0xD66A
9 | 0x0009 | 0xD77A | 0xD77B
PS D:\Desktop\CryptoAnalysis\crypto-analyze\lab2\task4>
  
```

Figure 8: 验证指导书密钥与明密文文件的对应关系

在已验证的明密文文件上运行密文筛选和半字节密钥枚举程序。筛选后保留 4028 对，去噪前正确对占比约 4.8%，去噪后占比

$$\frac{2^{16} \times 0.048}{4028} \approx 78\%$$

正确对占比显著提高。程序对目标半字节的 $2^4 = 16$ 种取值逐一计数。根据给定密钥 $K_6 = 0xBA98$ ，目标半字节为 $0x9$ 。它的计数达到最高值 3172，与错误候选 $0x8$ 并列。因此，本次枚举将正确的 4 bit 半字节包含在最高分候选中；当前计数结果还不能在 $0x8$ 和 $0x9$ 之间作出唯一判断。候选计数如下：

Table 2: 恢复目标半字节的候选计数

半字节候选	计数
0x8, 0x9	3172
0xC, 0xD	1902
0x0, 0x1	1544
0xA, 0xB	1468
0x2, 0x3	1268
0x4, 0x5	1232
0x6, 0x7	956
0xE, 0xF	198

```

PS D:\Desktop\CryptoAnalysis\crypto-analyze\lab2\task4> uv run python .\denoise_and_recover_k6.py
输入差分: 0x0001
四轮目标差分: 0x0010
输入差分明文对数: 65536
去噪后保留的明文对数: 4028

K6活动半字节猜测 | 计数
-----|-----
0x8 | 3172
0x9 | 3172
0xC | 1902
0xD | 1902
0x0 | 1544
0x1 | 1544
0xA | 1468
0xB | 1468
0x2 | 1268
0x3 | 1268
0x4 | 1232
0x5 | 1232
0x6 | 956
0x7 | 956
0xE | 198
0xF | 198

最高计数候选: ['0x8', '0x9']
参考密钥中K6对应半字节: 0x9
参考半字节候选排名: 2/16
PS D:\Desktop\CryptoAnalysis\crypto-analyze\lab2\task4>

```

Figure 9: 筛选后对目标半字节进行差分计数的结果

随后对五组新生成的六字密钥重复筛选实验。程序复用文件中的全部明文，但为每组六字密钥分别重新计算对应的五轮密文。任务一得到的四轮差分理论概率为 $p = 0.0479736328125$ ；当 $m = 2^{16}$ 时，理论正确对数为

$$mp = 2^{16} \times 0.0479736328125 = 3144.$$

设某组密钥下实际筛选保留 N_j 对，则用 mp/N_j 估计保留对中正确对所占比例。五组统计结果如下：

Table 3: 五组新密钥下的筛选保留数及正确对占比估计

组别	六字密钥组 ($K_1, \dots, K_6$)	实际保留数 N_j	mp/N_j
1	[0x0E6B, 0x0286, 0xAD7C, 0x496C, 0xFEDE, 0x93AE]	3906	80.49%
2	[0x36B5, 0xC640, 0x98AB, 0x1A0C, 0x095F, 0xF3C0]	3702	84.93%
3	[0xC0E4, 0x3F92, 0x4112, 0x1D08, 0x734C, 0xFB64]	3836	81.96%
4	[0x49A0, 0x7650, 0x9B9A, 0xAE68, 0xAA6B, 0x1255]	3696	85.06%
5	[0x8108, 0x5E1F, 0x13DD, 0xA3D8, 0xE75D, 0xB84D]	3656	86.00%

五组实际保留数的平均值为 3759.20 对，通过去噪阶段后正确对的平均占比为

$$\frac{2^{16} \times 0.048}{3759.20} \approx 84\%$$

```

Windows PowerShell
PS D:\DeskTop\CryptoAnalysis\crypto-analyze\lab2\task4> uv run python .\test_denoise_five_key_sets.py
输入明文数: 65536
输入差分: 0x0001
密文筛选差分: 0x0010, 0x0020, 0x0050
有序输入差分对数: 65536
理论差分概率p: 0.0479736328125, m*p: 3144.00
密钥组数: 5
随机种子: 20261003

序号 | 六字密钥组 | 实际保留数 | 理论正确对数m*p | 正确对占比估计
1 | [0x0E6B, 0x0286, 0xAD7C, 0x496C, 0xFEDE, 0x93AE] | 3906 | 3144.00 | 0.804916 (80.49%)
2 | [0x36B5, 0xC640, 0x98AB, 0x1A0C, 0x095F, 0xF3C0] | 3702 | 3144.00 | 0.849271 (84.93%)
3 | [0xC0E4, 0x3F92, 0x4112, 0x1D08, 0x734C, 0xFB64] | 3836 | 3144.00 | 0.819604 (81.96%)
4 | [0x49A0, 0x7650, 0x9B9A, 0xAE68, 0xAA6B, 0x1255] | 3696 | 3144.00 | 0.850649 (85.06%)
5 | [0x8108, 0x5E1F, 0x13DD, 0xA3D8, 0xE75D, 0xB84D] | 3656 | 3144.00 | 0.859956 (86.00%)

五组平均去噪保留对数: 3759.20
PS D:\DeskTop\CryptoAnalysis\crypto-analyze\lab2\task4>

```

Figure 10: 五组新密钥下的去噪保留数及正确对占比估计

7.1.2 复杂度分析

- 数据复杂度: 选择了 2^{16} 个明文对, 即 2^{17} 个明文。
- 时间复杂度:
 - 采样阶段要获得 2^{17} 个明文对应的密文, 即做 2^{17} 次 5 轮 CipherFour 算法的加密运算;
 - 去噪阶段计算 2^{16} 个密文对的差分, 即 2^{16} 次异或运算;
 - 恢复密钥阶段: 去噪后保留 ϵn 对密文, 每个 4 bit 半字节有 2^4 种猜测。对每一种猜测, 都需要逐一检查全部保留对, 因此需要 $2^4 \times \epsilon n$ 次差分方程验证。本次保留 4028 对, 对应 $16 \times 4028 = 64448$ 次验证。
- 存储复杂度:
 - 采样阶段存储选择明文对应的密文, 即 $2^{17} \times 16 = 2^{21}\text{bit} = 256\text{KB}$;
 - 4bit 每种设置计数器, 若每个计数器占 8bit, 则计数器的存储复杂度约为 $2^4 \times 8 = 2^7\text{bit}$ 。

8 实验总结

A 实验代码

本附录列出本次实验中使用的 Enigma 加密及性能测试程序。

A.1 寻找差分路线

```
1  """
2  练习3.2 CipherFour算法的4轮最优差分路线的搜索程序
3
4  搜索思路：
5      (1) 首先计算S盒的差分分布表DDT.
6      (2) 对所有 $2^{16}-1$ 个非零输入差分同时进行搜索.
7      (3) 每一轮依次计算4个S盒的最大传播概率，再进行P置换.
8      (4) 保存每轮到达各个差分状态的最大概率，最后倒推出完整路线.
9
10  注意：本程序搜索的是概率最高的单条差分路线(differential trail)，不是将相同输入，
11      输出差分之间的多条路线概率相加.
12  """
13  import numpy as np
14
15
16  # CipherFour算法的S盒定义
17  S = [9, 8, 0, 1, 14, 11, 12, 13, 3, 2, 5, 7, 15, 10, 6, 4]
18
19  # CipherFour算法的P置换定义
20  P = [0, 4, 8, 12, 1, 5, 9, 13, 2, 6, 10, 14, 3, 7, 11, 15]
21
22
23  def diff_T():
24      """计算S盒的差分分布表DDT."""
25      diff_table = [[0 for _ in range(16)] for _ in range(16)]
26
27      # 遍历所有输入差分
28      for diff_in in range(16):
29          # 遍历S盒的所有输入
30          for x1 in range(16):
31              x2 = x1 ^ diff_in
32              diff_out = S[x1] ^ S[x2]
33              diff_table[diff_in][diff_out] += 1
34
35      return diff_table
36
37
38  def P_box(value):
39      """对16比特差分状态进行P置换."""
40      bin_value = bin(value)[2:].zfill(16)
41      output = ""
```

```
42
43     for i in range(16):
44         output += bin_value[P[i]]
45
46     return int(output, 2)
47
48
49 def split_nibbles(value):
50     """将16比特状态拆成4个4比特差分."""
51     return [
52         (value >> 12) & 0xF,
53         (value >> 8) & 0xF,
54         (value >> 4) & 0xF,
55         value & 0xF,
56     ]
57
58
59 def update_one_sbox(score, probability_table, position):
60     """
61     对指定位置的一个S盒进行差分传播.
62
63     score是一个16*16*16*16的数组，四个下标分别表示
64     一个16比特差分的四个 半字节.
65     """
66     new_score = np.zeros_like(score)
67
68     for diff_in in range(16):
69         for diff_out in range(16):
70             probability = probability_table[diff_in][diff_out]
71
72             # DDT中计数为0，说明该传播不可能
73             if probability == 0:
74                 continue
75
76             # 对当前半字节取最大路线概率，因为P置换前4个部分是相互独立的，可以分
77             # 别最大化
78             if position == 0:
79                 candidate = score[diff_in, :, :, :] * probability
80                 new_score[diff_out, :, :, :] = np.maximum(
81                     new_score[diff_out, :, :, :], candidate
82                 )
83             elif position == 1:
84                 candidate = score[:, diff_in, :, :] * probability
85                 new_score[:, diff_out, :, :] = np.maximum(
86                     new_score[:, diff_out, :, :], candidate
87                 )
88             elif position == 2:
89                 candidate = score[:, :, diff_in, :] * probability
90                 new_score[:, :, diff_out, :] = np.maximum(
```

```

90         new_score[:, :, diff_out, :], candidate
91     )
92     else:
93         candidate = score[:, :, :, diff_in] * probability
94         new_score[:, :, :, diff_out] = np.maximum(
95             new_score[:, :, :, diff_out], candidate
96         )
97
98     return new_score
99
100
101 def search_best_route(pos):
102     """搜索4轮CipherFour的全局最优单条差分路线."""
103     diff_table = diff_T()
104
105     # 将DDT计数转换为S盒差分传播概率
106     probability_table = []
107     for diff_in in range(16):
108         row = []
109         for diff_out in range(16):
110             row.append(diff_table[diff_in][diff_out] / 16.0)
111         probability_table.append(row)
112
113     # 提前计算全部16比特状态的P置换结果
114     permutation = np.array([P_box(x) for x in range(2 ** 16)])
115
116     # best_probability[r][d]表示经r轮后到达差分d的最大单条路线概率
117     best_probability = []
118     # 第一个索引是看哪一轮，第二个索引看输出差分
119     # 允许所有非零输入差分，初始概率设为1
120     first_round_input = np.ones(2 ** 16)
121     first_round_input[0] = 0.0 #输入差分为0直接把概率设成0，这样不用考虑这条路径了
122     if pos == 0:
123         first_round_input[1] = 0.0
124     best_probability.append(first_round_input)
125     # 向前计算4轮的最大概率
126     for round_index in range(4):
127         score = best_probability[round_index].reshape(16, 16, 16, 16) #把输出差分分成4个部分，16bit每个部分4bit也就是0-15,score就是B_r
128
129         # 依次处理4个并行S盒
130         for position in range(4):
131             score = update_one_sbox(score, probability_table, position)
132
133         after_sbox = score.reshape(2 ** 16) #再弄回一整个输出差分
134
135         # 将S盒输出差分经过P置换
136         after_pbox = np.zeros(2 ** 16)

```

```
137     after_pbox[permutation] = after_sbox #让after_pbox的第P[i]个位置等于
after_sbox[i]
138     best_probability.append(after_pbox) #得到best_probability[round_index
+ 1]
139     if round_index + 1 == pos:
140         best_probability[round_index + 1][1] = 0.0 #禁用输入差分0001
141
142     # 找到4轮后概率最大的输出差分
143     current_difference = int(np.argmax(best_probability[4]))
144     route = [current_difference]
145     round_probabilities = []
146
147     # 从第4轮向第1轮倒推最优路线
148     for round_index in range(4, 0, -1):
149         # P是对合置换, 所以再过一次P即可得到S盒输出
150         sbox_output = P_box(current_difference)
151         output_nibbles = split_nibbles(sbox_output)
152
153         best_predecessor = 0
154         best_value = -1.0
155         best_one_round_probability = 0.0
156
157         # 遍历上一轮的全部差分状态
158         for predecessor in range(2 ** 16):
159             predecessor_probability = best_probability[round_index - 1][
predecessor] #找上一轮的概率
160             if predecessor_probability == 0:
161                 continue
162
163             input_nibbles = split_nibbles(predecessor)
164             one_round_probability = 1.0
165
166             for position in range(4):
167                 one_round_probability *= probability_table[input_nibbles[
position]][output_nibbles[position]]
168             #每个S盒变换概率乘起来
169             candidate = predecessor_probability * one_round_probability
170
171             if candidate > best_value:
172                 best_value = candidate
173                 best_predecessor = predecessor
174                 best_one_round_probability = one_round_probability
175
176             route.append(best_predecessor)
177             round_probabilities.append(best_one_round_probability)
178             current_difference = best_predecessor #把输出差分变成刚刚找到的前驱,
然后重新来
179
180     route.reverse()
```

```
181     round_probabilities.reverse() #倒过来就是输入到输出了
182
183     total_probability = 1.0
184     for probability in round_probabilities:
185         total_probability *= probability #每一轮乘起来
186
187     return route, round_probabilities, total_probability
188
189
190 if __name__ == "__main__":
191     for pos in range(5):
192         print(f"第{pos}轮不为0001")
193         route, round_probabilities, total_probability = search_best_route(pos
194 )
195
196         print("CipherFour算法的4轮最优差分路线为:")
197         print(" -> ".join(hex(x)[2:].zfill(4).upper() for x in route))
198         print("各轮概率为:", round_probabilities)
199         print("总概率:", total_probability, "= 2^" + str(int(np.log2(
200 total_probability))))
201         print()
```

Listing 1: .py

B 实验报告完整地址

实验完整代码、实验报告 L^AT_EX 源代码见仓库: <https://github.com/infinitepwn/crypto-analyze.git>